



UNIVERSITY OF PISA
DEPARTMENT OF COMPUTER SCIENCE

Master programme in data science and business informatics

Decision Support Systems:
LABORATORY OF DATA SCIENCE

Professor:
Anna Monreale

Group:
Truong Manh Nguyen
Thi Hien Do

Academic Year 2025/2026

CONTENTS

INTRODUCTION	1
Chapter 1: Using Python	2
1.1. Data Understanding and Cleaning	2
Assignment 1-2	2
1.2. Profiling	3
Assignment 3	3
1.3. Data Warehouses design, data splitting and populating	5
Assignment 4: DW Schema	5
Assignment 5: Data splitting and preparation of warehouse-ready tables	6
Assignment 6: Data uploading into the data warehouse using Python	6
Chapter 2: SQL Server Integration Services	7
Assignment 7	7
Assignment 8	8
Assignment 9	8
Assignment 10	9
Assignment 11	9
Assignment 12	11
Assignment 13	12
Chapter 3: Using MDX, SQL Management Studio, and PowerBI	12
Assignment 14	12
Assignment 15	13
Assignment 16	13
Assignment 17	14
Assignment 18	14
Assignment 19	15
Assignment 20	15
Assignment 21	16
Assignment 22	17

INTRODUCTION

This report presents an end-to-end data analytics workflow for a music streaming context, moving from raw data to a Data Warehouse, ETL, and OLAP/BI to enable analytical querying and actionable insights.

In Chapter 1, we perform exploration, cleaning, and enrichment on two datasets: artists and tracks. For artists, we identify key characteristics (gender imbalance and strong concentration in the Italian scene) and address missing values through external-source enrichment, combined with geocoding/reverse geocoding (OpenStreetMap) and Regex-based extraction to infer `active_start`, nationality, and a new `is_group` attribute. For tracks, we remove duplicates, standardize the explicit label, and implement a structured imputation framework, while also creating derived lyric-based features. We then conduct two complementary profiling layers: (i) melody-based clustering using K-means on acoustic features, and (ii) lyrics topic modeling to assign four interpretable content topics.

Building on these outputs, Chapter 1 also designs and prepares the Data Warehouse, followed by automated loading into SQL Server using Python with a controlled and performance-aware ingestion pipeline.

In Chapter 2, we implement ETL pipelines to sample the fact table and load the corresponding dimension subset, and we develop analytical workflows including yearly artist ranking, summer–winter comparisons by region, regional stream-share by category, and a Trending vs Flopping model.

Finally, Chapter 3 builds the OLAP cube, executes MDX queries, and delivers Power BI visualizations to summarize trends and insights supporting release planning, marketing, and content strategy.

Chapter 1: Using Python

1.1. Data Understanding and Cleaning

Assignment 1-2

The dataset **artists** contains 104 records and 17 attributes describing artist-related information includes a unique artist identifier *id_author*, *name*, and attributes such as *gender*, birth information, *nationality*, geographic fields like *birth_place*, *province*, *region*, *country*, coordinates, and a short textual *description*.

An exploratory analysis reveals a strong gender imbalance, with male artists largely prevailing over female artists. The dataset is highly localized to the Italian music scene; most artists hold Italian nationality and are concentrated in specific regions, particularly Lombardia, Campania, and Lazio. The age distribution shows that artists are predominantly concentrated in the 30–45 age range. Textual analysis of the description field using CountVectorizer identified "rapper" and "italiano" as the most frequent keywords confirming that the dataset mainly represents artists belonging to the Italian rap and hip-hop scene.

While the dataset contains no duplicates, it presents a substantial amount of missing data across several attributes. Notably, *active_start* is missing for 51.9% of the artists, while *active_end* is entirely missing (100%), as many artists are currently active. Biographical and geographic attributes such as *birth_date* (29.8%), *birth_place* (30.8%), *nationality* (31.7%), *province* (32.7%), *region* (34.6%), *country* (32.7%), *latitude* (30.8%), and *longitude* (30.8%). The description field is missing for 17.3% of the records.

To mitigate this issue, we implemented a data enrichment strategy by **integrating external sources** such as **Wikipedia** to fill missing attribute values whenever information was available. Subsequently, we employed **OpenStreetMap's Nominatim** geocoder through the *geopy* library to recover the remaining missing geographic information. Specifically, reverse geocoding was applied to existing geographic coordinates (latitude and longitude) in order to infer and populate missing *city*, *province*, *region*, and *country* fields. In addition, forward geocoding was performed using the *birth_place* attribute to retrieve and populate *birth_province*, *birth_region*, *birth_country*, as well as the corresponding geographic coordinates. For Italian-born artists, verified birth data is used to backfill missing primary geographic attributes. Furthermore, we used **Regular Expressions (Regex)** on the description field to extract career *active_start* dates, infer *nationality*, and create a new *is_group* attribute to distinguish between solo artists and collectives.

The enrichment process improved data completeness across all key attributes. Missing values for *nationality* and *country* dropped to 9.6%, an improvement of over 22%. Geographic fields, including *region*, *province*, and coordinates, saw recoveries of 15–18% via geocoding, while *active_start* dates were recovered for nearly 20% of the artists.

The dataset **tracks** comprises a total of 11,166 instances across 37 distinct attributes, representing various characteristics of musical tracks. The data types detected within the dataset include three integer-type attributes, twenty attributes with floating-point values, and fourteen attributes in textual format. Preliminary inspection revealed the presence of four duplicate records, all associated with the same primary_artist, namely *Madame*. To preserve data quality, these records were de-duplicated by retaining only the instance exhibiting the highest value in the *stream@1month* attribute.

Subsequent analysis of the *explicit* attribute revealed a nearly balanced distribution, with 51.1 percent of tracks labelled as explicit and 48.9 percent as *non-explicit* with *swear words* range from 0 to 72 (for both EN and IT). A visual representation using a correlation matrix and several bar charts was implemented to assess inter-attribute relationships; however, no substantial or statistically significant correlations were observed. To gain further insight, the distributions of several *melodic* and *spectral* attributes were analysed. Specifically, attributes such as *bpm*, *rolloff*, *flux*, *rms*, *flatness*, *spectral_complexity*, *pitch*, and *loudness* were examined. The findings indicated that most features exhibit non-normal distributions, being either left-skewed or right-skewed. Exceptions to this pattern included *spectral_complexity* and *pitch*, which appeared to follow a distribution closely approximating the normal curve.

A significant proportion of attributes displayed missing values. Out of 37 attributes, 28 contained missing entries. The attribute *lyrics* demonstrated the lowest proportion of missing data with only three absent values, whereas *featured_artists* exhibited the highest missing rate at 68.5 percent, corresponding to 7649 records. In order to enhance data completeness and maintain analytical integrity, a structured imputation framework was implemented.

Initially, for the pair of attributes *album* and *album_name*, a cross-imputation approach was adopted whereby the non-missing value in one attribute was used to populate its counterpart. If both attributes were missing, the corresponding entry was retained as null. With regard to *featured_artists*, null values were interpreted as instances of solo tracks performed solely by the *primary_artist*, and thus left unaltered. The negligible proportion of missing lyrics values (only 3 records) were assigned a NaN placeholder to maintain consistency.

Considering the earlier observation that most *melodic* features were skewed, it was deemed appropriate to apply the median value as a robust imputation strategy for the eight associated attributes. In contrast, attributes such as *disc_number*, *track_number*, *popularity*, and *album_type* were left unaltered in the presence of missing data. For missing *id_album* entries, a placeholder value of -1 was used to denote the absence of identifiable album information.

For the *language* attribute, a heuristic approach was adopted whereby language detection was performed using the lyrics text. In the rare case of tracks lacking lyrics (three instances), the language attribute was left missing.

Additional inconsistencies in the *explicit* attribute were addressed by verifying the presence of offensive language within the lyrics. If a track contained profane content yet was not labelled as explicit, its label was revised to ensure consistency. Conversely, tracks flagged as explicit but lacking offensive terms were reviewed and, if necessary, reclassified.

In cases where attributes such as *year*, *month*, and *day* were missing, values were extracted from the *album_release_date* to fill these gaps. Finally, several derived features were computed from the *lyrics* field, including *char_per_tok*, *n_tokens*, *n_sentences*, and *avg_token_per_clause*. For records without lyrics, these derived attributes were imputed with a value of zero to maintain numerical consistency within the dataset.

1.2. Profiling

Assignment 3

The dataset was profiled along two complementary analytical perspectives, namely melody-based profiling and lyrics-based profiling. This dual strategy was adopted to capture both acoustic and semantic heterogeneity across the track catalogue.

Melody-based profiling

For melody-based profiling, eight acoustic attributes that characterise melodic and spectral properties were selected for clustering analysis: *bpm*, *rolloff*, *flux*, *rms*, *flatness*, *spectral_complexity*, *pitch*, and *loudness*. The elbow method was applied with K-means to determine an appropriate number of clusters. Based on this diagnostic, the solution with $k = 4$ was selected as the most suitable configuration, yielding a silhouette score of 0.1707. Cluster structure was subsequently visualised using t-distributed stochastic neighbour embedding (t-SNE), and cluster characteristics were interpreted through their centroid profiles.

On the basis of centroid patterns, the four clusters were assigned descriptive labels reflecting their dominant acoustic signatures:

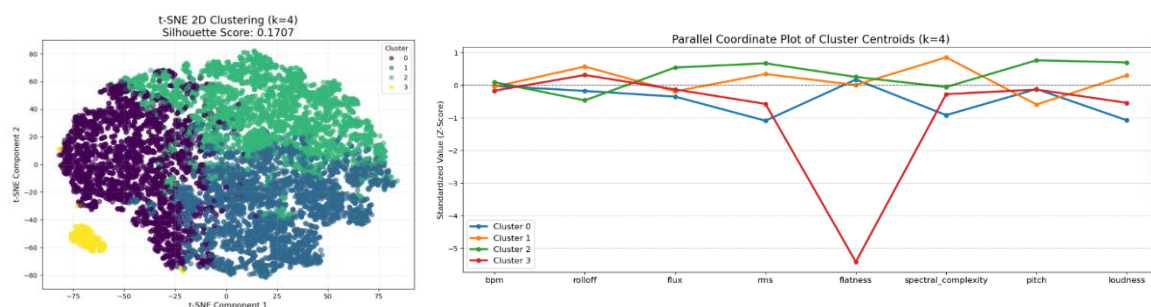


Figure 1: t-SNE and Cluster Centroids visualization

- **Cluster 0 (Blue) - Pure Tonal:** This cluster is characterised by extremely low flatness (approaching a Z-score of -5), suggesting a highly tonal, harmonically coherent, and acoustically “clean” profile.
- **Cluster 1 (Orange) - High Energy:** This cluster exhibits elevated values of flux, root mean square energy, pitch, and loudness, indicating tracks with intense dynamics and energetic delivery, which may be consistent with high-intensity genres such as rock and electronic dance music.
- **Cluster 2 (Green) - Mellow:** This cluster presents below-average values across most features, particularly root mean square energy, spectral complexity, and loudness. The resulting profile appears consistent with quiet or background-oriented music, including lo-fi or ambient-like production.
- **Cluster 3 (Red) - Complex:** This cluster is defined by the highest spectral complexity and relatively high rolloff, which together suggest dense spectral content and greater high-frequency distribution.

Lyrics-based profiling

For lyrics-based profiling, topic modelling was conducted using the *pyLDAvis* framework to support interpretability. Lyrics content was analysed and partitioned into four latent topics, each represented by the top 50 most salient tokens, as follows:

Topic 1: lyrics, bitch, love, like, know, fuck, gang, go, one, get, bla, back, money, life, shit, dark, bad, time, let, sick, big, bla bla, make, top, bling, luke, flow, narcos, sick luke, say, noyz, trap, real, slime, two, way, feel, cause, tony, night, bitches, need, keep, give, good, new, boy, check, crazy, king

Topic 2: ca, pe, int, nn, cu, cchiù, tutt, mo, pecché, comme, song, stu, comm, dint, sî, sempe, sulo, aggio, ch, nuje, maje, fà, hê, quanno, po, staje, ncopp, sorde, ij, ccà, addò, songo, chesta, ammore, chello, prima, stongo, core, simmo, tiempo, fa, mmé, sti, vo, faje, saccio, ro, ra, vota, fá

Topic 3: uoh, yah, ehi, skrrt, mhm, gang, bu, uoh uoh, pow, brr, pa, uah, mhm mhm, ehi ehi, ahi, bang, bu bu, yah yah, okay, pah, pam, skrrt skrrt, money, ra, gang gang, pa pa, ahi ahi, bang bang, pow pow, bam, yeh, grr, bitch, pam pam, uah uah, skrt, pah pah, brr brr, vroom, ra ra, cash, milano, yay, squad, bam bam, money money, yeh yeh, drin, trap, woo

Topic 4: solo, mai, poi, sempre, fa, eh, sai, ora, ogni, senza, fare, vita, ancora, così, cosa, fra, sì, dentro, voglio, mondo, vuoi, niente, fuori, fatto, cazzo, bene, va, già, qua, prima, tempo, po, male, casa, testa, due, ehi, adesso, gente, via, soldi, giorno, puoi, dopo, rap, amore, tipo, altro, meglio, sento

Based on the semantic coherence suggested by the dominant tokens in each topic, the topics were labelled as follows:

- **Topic 1** → **Trap Gang Life**
- **Topic 2** → **Neapolitan Rap**
- **Topic 3** → **Street Trap Vibes**
- **Topic 4** → **Italian Rap Life**

This profiling stage therefore provided two interpretable segmentation layers: an acoustic segmentation derived from clustering of melodic attributes and a semantic segmentation derived from probabilistic topic modelling of lyrical content.

1.3. Data Warehouses design, data splitting and populating

Assignment 4: DW Schema

Our goal is to design a data warehouse that supports analytical queries for a music streaming platform. We defined the grain of fact table, `Published_song_fact`, as one published song record. The main measure stored is the number of streams one month after release, which is an additive measure and can be aggregated across all dimensions. The duration attribute is treated as a semi-additive measure. Attributes such as `title`, `track_id`, `track_number`, and `explicit` are modeled as degenerate dimensions, as they directly describe the song record and do not require a separate dimension table.

The fact table contains foreign keys referencing the dimension tables, which provide the descriptive context required for analysis. We defined the following dimensions:

- **Date**: enables temporal analysis by decomposing the song release date into hierarchical levels such as *day*, *month*, *quarter*, and *year*.
- **Album**: provides album-level information like *album_name*, and *album_release_date*.
- **Artist**: contains artist-related attributes like *artist_name*, *gender*, *birth_date*, *is_group*. This dimension is snowflaked with the **ArtistGeography** dimension and includes two foreign keys, *current_geo_id* and *birth_geo_id*, allowing analysis based on both the artist's current location and birthplace. It also is designed to support type 2 Slowly Changing Dimensions with *id_author* as the natural key of the artist, along with *active_start* and *active_end* attributes to track the period of the artist's musical activity.
- **ArtistGeography**: stores geographic information such as *city*, *region*, and *country*, *h3* supporting spatial analysis.
- **SongArtist**: acts as an auxiliary table to handle the many-to-many relationship between songs and artists, allows distinguishing between primary and featured artists.
- **Category**: modeled as a junk dimension, grouping low-cardinality categorical attributes derived from melodic and lyrical profiling.
- **Lyrics**: stores the textual content of song lyrics and is kept separate due to its large storage requirements.

All dimensions are identified by integer surrogate keys, except for the Date dimension, which uses a *date_id* in the format `YYYYMMDD`. To ensure referential integrity and handle missing values consistently, a default value of `-1` was assigned whenever a primary key reference was null.

The detailed structure of the data warehouse schema is shown in the figure below

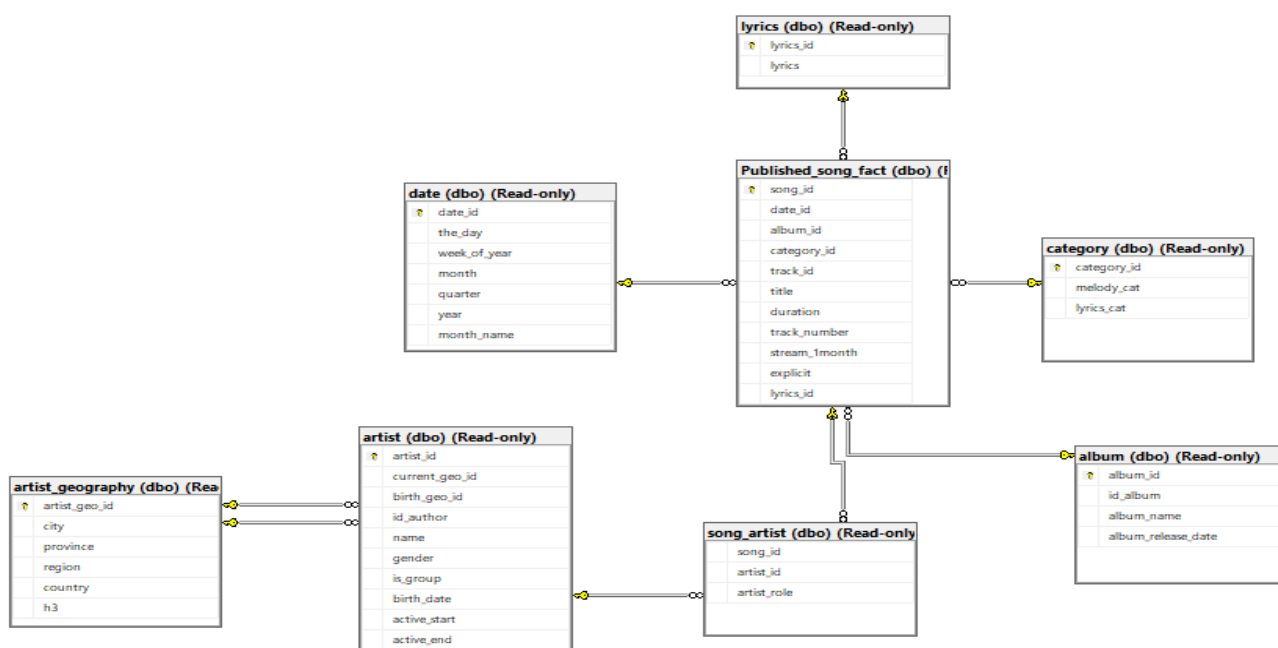


Figure 2: Data Warehouses design

Assignment 5: Data splitting and preparation of warehouse-ready tables

Specifically, the Album and Category dimensions were de-duplicated using a lookup-map mechanism (dictionary-based key–value mapping), thereby reducing redundancy and improving consistency of categorical representations. In addition, the Lyrics table was designed using a vertical partitioning strategy, whereby high-volume textual content was separated from the central fact table in order to minimise storage pressure and avoid unnecessary I/O overhead during analytical queries. The Date dimension was enriched with derived temporal attributes, including quarter and week-of-year, to support multi-level time-based aggregation. In parallel, the Artist dimension was cleaned and standardised from the XML source and subsequently enriched by automatically incorporating newly detected featured artists extracted from the JSON source. Finally, the central fact table was constructed by mapping foreign keys to the corresponding dimensions and standardising measure fields for analytical consistency. A dedicated bridge table was also generated to explicitly represent artist participation roles.

Assignment 6: Data uploading into the data warehouse using Python

The data loading process into the data warehouse was fully automated in Python using the *pyodbc* library. The workflow commenced with a strict database sanitation strategy, including temporary deactivation of foreign-key constraints, removal of pre-existing records, and reseeding of identity counters to ensure a clean integration environment. Prior to ingestion, the CSV outputs generated during the preparation stage were passed through a standardisation filter to enforce consistent date formatting (YYYY-MM-DD) and to harmonise missing values as SQL-compatible NULL representations, thereby improving compatibility with Microsoft SQL Server.

To address performance constraints typically observed during large-scale inserts, the loading pipeline employed batch processing with a commit size of 5,000 rows per transaction. This batching strategy reduced network overhead, limited transaction log growth, and improved execution efficiency. The insertion sequence was executed in dependency-aware order: dimension tables were loaded first to establish referential targets, followed by the central fact table, and finally the bridge table. This ordering strategy ensured that referential integrity was preserved throughout the ingestion process.

Chapter 2: SQL Server Integration Services

Assignment 7

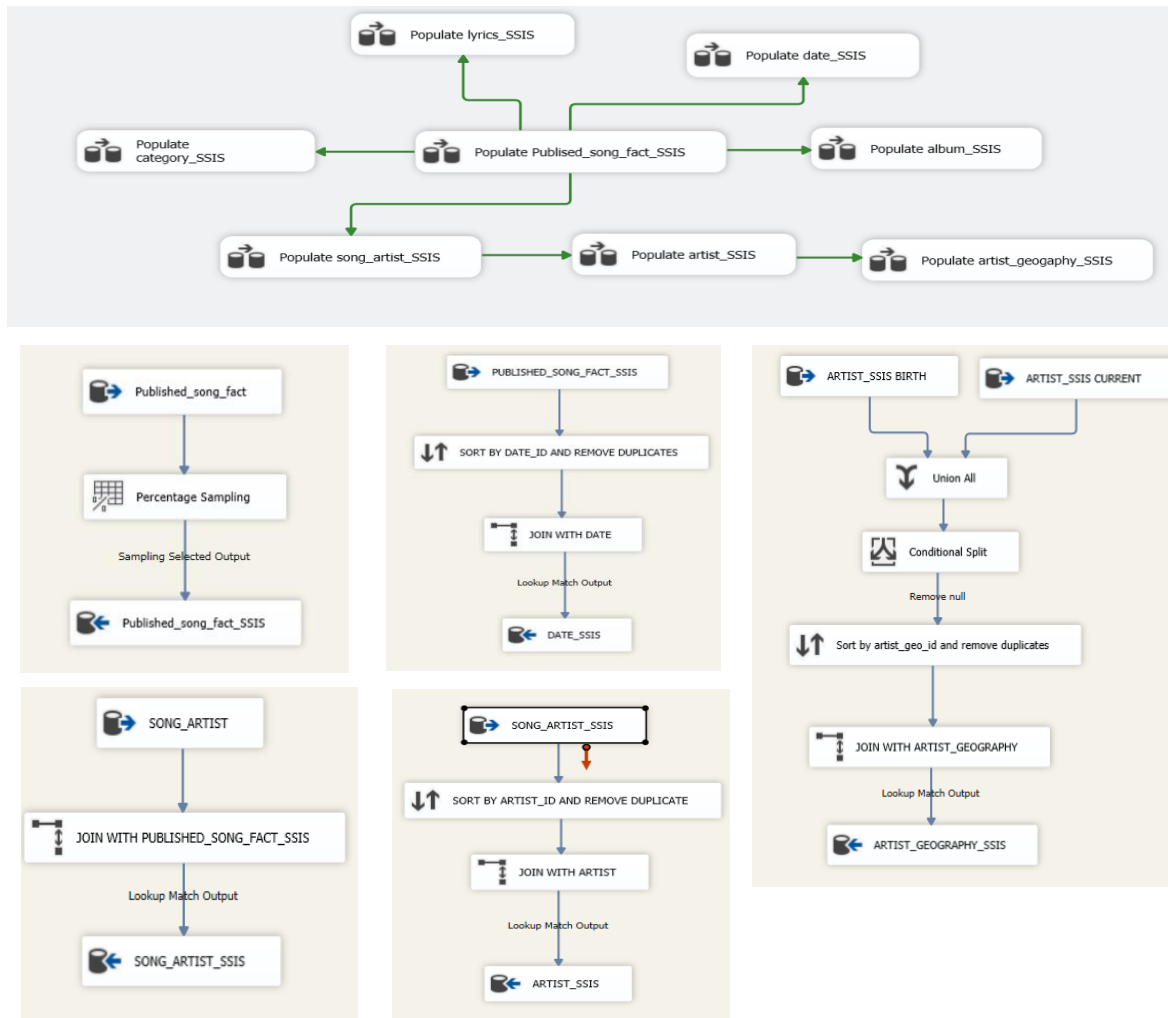


Figure 3: Assignment 7 flow

We created an SSIS project composed of multiple Data Flow tasks, one for each table, and designed a control-flow order that respects dependencies between tables.

We started from `Published_song_fact` and applied Percentage Sampling to extract 30% of the rows. The sampled output was inserted into `Published_song_fact_SSIS`. After that, we populated the related tables `Date_SSIS`, `Category_SSIS`, `Album_SSIS`, and `Lyrics_SSIS` by using `Published_song_fact_SSIS` as the reference filter for distinct foreign keys and applying Lookups on the original dimension tables to load only the records whose keys matched the sampled fact data.

To populate `Song_artist_SSIS`, we selected all attributes `song_artist` table included `song_id`, `artist_id`, and `artist_role` and used a Lookup to join with `Published_song_fact_SSIS` by matching on `song_id`.

To populate `Artist_SSIS`, we started from `Song_artist_SSIS`, selected, sorted by `artist_id` and removed duplicates, then used a Lookup on the original `artist` table to load only the artists appearing in the sampled relationships.

We populated `Artist_geography_SSIS` starting from `Artist_SSIS` by creating two separate data sources: one selected `birth_geo_id` and the other selected `current_geo_id`. These two sources were then merged using a Union All into a single column `artist_geo_id`. After that, we applied a Conditional Split to remove records with null geography identifiers and used a Sort transformation to eliminate duplicate `artist_geo_id` values. Finally, we performed a Lookup on the original `artist_geography` table to load the corresponding geography records into `Artist_geography_SSIS`.

In the following analysis, we assume that streaming performance should be primarily attributed to the *primary artist* of a song. To avoid double counting and ensure consistency across the SSIS workflows, OLAP cube, and MDX queries, streams are either fully credited to the primary artist or allocated according to artist roles in the case of collaborations, as specified in the corresponding assignments. This assumption reflects common industry practices and provides a coherent basis for evaluating artist performance throughout the analytical process.

Assignment 8

We started from the `song_artist` source selected `song_id`, `artist_id` and `artist_role` and then applied Conditional Split to keep only the Primary artists. Then we used a Lookup on `published_song_fact` with match on `song_id` to get `date_id`. With a second Look Up on the date dimension with match on `date_id`, we retrieved the year. We removed rows where year is NULL using a Conditional Split. Next, we performed a Lookup on the artist dimension with match on `artist_id` to get the artist name. Finally, we Aggregated by year and artist and computed `COUNT(song_id)` as `total_songs_published`, then Sorted by year and `total_songs_published` (DESC) to rank artists within each year. In accordance with the initial assumption, only songs where the artist plays a primary role were counted.

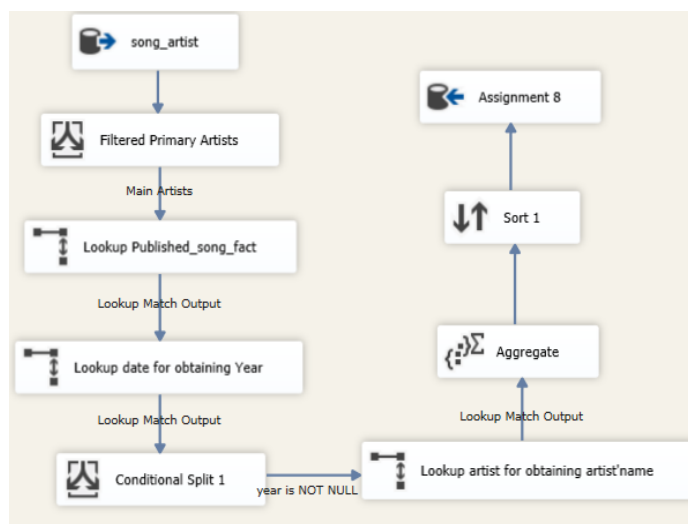


Figure 4: Assignment 8 data flow

Assignment 9

We loaded data from `song_artist` selected `song_id`, `artist_id` and `artist_role` and retained only records corresponding to primary artists. Then we used a Lookup on `published_song_fact` to add `date_id` and `stream_1month`, and a Lookup on the date table to get the month of songs release. We used Conditional Split to filtered the rows to keep only summer months 6, 7, 8 and winter months 12, 1, 2. After that, we used a Lookup on the artist table by matching `artist_id` to retrieve `current_geo_id`, and then a Lookup on artist geography by matching `artist_geo_id` to obtain the region. We removed rows where region is null using a Conditional Split.

In Derived Columns Summer–Winter Measures, we created four new columns:

- `summer_songs` = 1 if month is 6, 7, or 8, otherwise 0
- `winter_songs` = 1 if month is 12, 1, or 2, otherwise 0
- `summer_streams` = `stream_1month` if month is 6, 7, or 8, otherwise 0
- `winter_streams` = `stream_1month` if month is 12, 1, or 2, otherwise 0

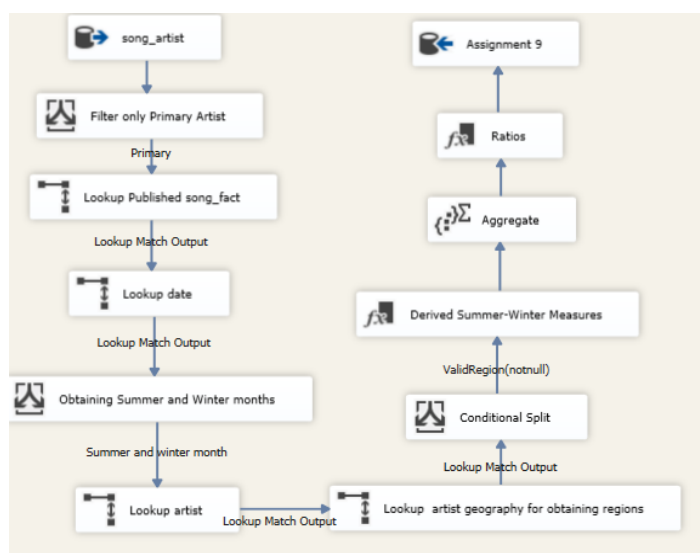


Figure 5: Assignment 9 data flow

We then Aggregated by *region*, summing up these measures to get total summer songs, total winter songs, total summer streams, and total winter streams. Finally, we computed ratios:

- Song summer-winter score = total summer_songs / total winter_songs
- Stream summer-winter score = total summer_streams / total winter_streams

Assignment 10

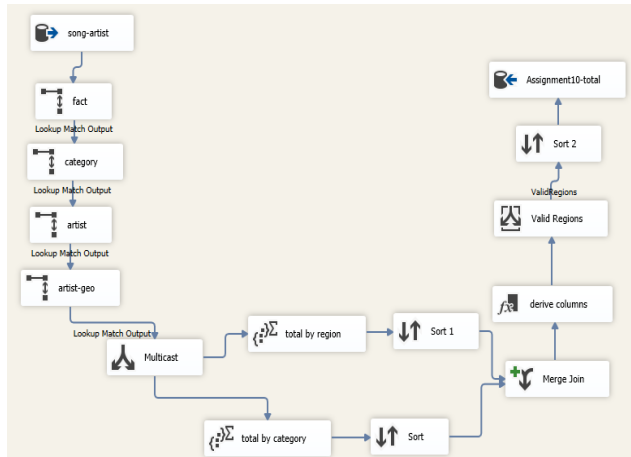


Figure 6: Assignment 10 data flow

A single Data Flow Task was implemented in the Control Flow panel using an OLE DB Source configured on the song_artist table. At the extraction stage, only rows corresponding to artists with the role *Primary* were selected. The resulting dataset was subsequently enriched through four Lookup transformations in sequence, which were used to retrieve the relevant analytical attributes from Published_song_fact, category, artist, and artist_geography, respectively. Following enrichment, a Multicast transformation was applied to split the data flow into two parallel branches. In the first branch, an Aggregate transformation was used to calculate

regional_streams, defined as the total streams for each region within each melody category. In the second branch, a separate Aggregate transformation was applied to compute total_cat_streams. Both aggregated outputs were then sorted by melody_cat and then merge to re-associate each region-level total with its corresponding category-level total.

Two additional measures: other_regions_streams and stream_ratio_percentage was derived to operationalise the ratio specified by the task requirements. Records with null geographic regions were removed by Conditional Split. Finally, the resulting dataset was sorted according to the required analytical ordering and exported into the target database through an OLE DB Destination. The ratio metric was computed using the following formulation:

$$\text{stream_ratio_percentage} = \frac{\text{regional_streams}}{\text{total_cat_streams} - \text{regional_streams}}$$

Figure 7: Assignment 10's ratio metric

Assignment 11

A single Data Flow Task was configured by integrating *primary-artist* participation via Conditional Split. The filtered stream was then enriched through a sequence of Lookup operations against the fact, Date and Artist table. Subsequently, a Multicast transformation duplicated the enriched stream to support parallel computation paths. In one branch, an Aggregate transformation produced an artist-level summary to rank their song based on the released date of songs, which was then sorted (Sortartist). In the second branch, the non-aggregated stream was also sorted (Sortartist2). Both sorted outputs were reconciled through a Merge Join, after which a Derived Column (IsFirstSong) was introduced to flag records corresponding to the artist's debut track. The resulting dataset was then split via Multicast 1.

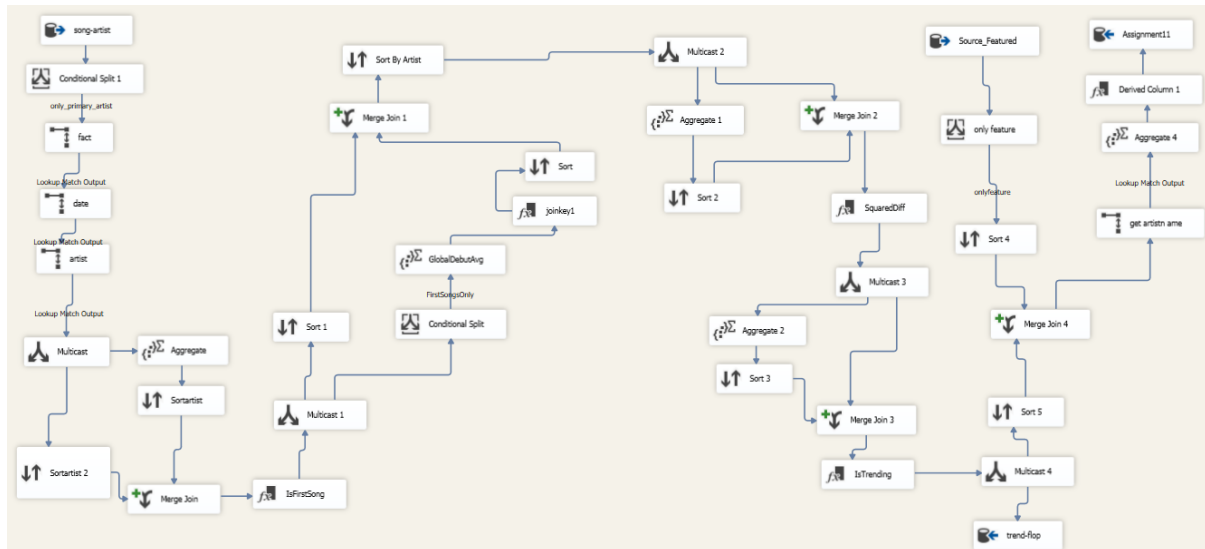


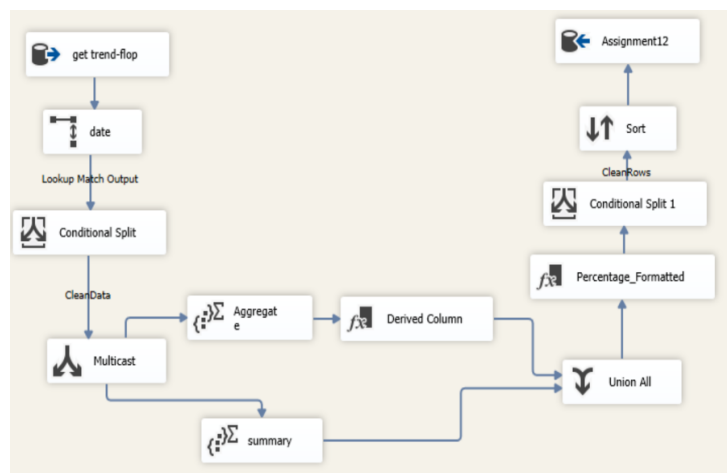
Figure 8: Assignment 11 data flow

The stream was filtered using a Conditional Split (*FirstSongsOnly*), retaining only debut-track rows. An Aggregate step (*GlobalDebutAvg*) was applied to calculate the overall debut average across all artists and also count the total number of debut song. Because this global statistic had to be propagated back to each artist record, a Derived Column (*joinkey1*) was created to define a constant join key, followed by a Sort operation to satisfy merge constraints. The global baseline and the per-artist stream were subsequently integrated through Merge Join 1 (based on dummy key), and the merged output was sorted by artist (*Sort By Artist*) to enable downstream joins.

The mean and normalised deviation logic was then implemented via Multicast 2, which split the sorted dataset into two branches. In the first branch, an Aggregate transformation (*Aggregate 1*) generated all artist average streams and their total songs; the output was then sorted (*Sort 2*) by *artist_id*. In the second branch, the detailed dataset was retained and merged with the aggregated metrics through Merge Join 2. A Derived Column (*SquaredDiff*) was subsequently applied to compute a squared deviation term. The resulting dataset was duplicated again using Multicast 3. One branch was aggregated (*Aggregate 2*) and sorted (*Sort 3*) to construct a *sum-square-diff* by *artist_id*, while the other branch retained row-level detail; both were reconciled through Merge Join 3. A final Derived Column (*IsTrending*) operationalised the classification rule, producing a categorical indicator distinguishing trending versus flopping behaviour based on the rule in the requirement.

The output stream was finally duplicated through Multicast 4. One branch was written directly to an OLE DB Destination (*trend-flop*) as a consolidated trending/flop dataset to support the assignment 12 calculation. In parallel, the second branch was sorted (*Sort 5*) by *song_id* and merged with a secondary pipeline extracted from *Source_Featured*, which was filtered using a Conditional Split (*only feature artist from song_artist*) and sorted (*Sort 4*) by *song_id* before being joined via Merge Join 4. This integration ensured that the final reporting layer incorporated featured-artist participation where required. A subsequent Lookup (*get artist name*) was applied to obtain artist name, followed by Aggregate 4 and a final Derived Column 1 to compute and format the deliverable measures (trending percentage and trending factor). The refined output was then persisted to the designated destination (*Assignment11*) through an OLE DB Destination.

Assignment 12



$$\text{Percentage} = \frac{\text{TrendingCount}}{\text{TotalReleased}}$$

Figure 9: Assignment 12 data flow and metric

Assignment 12 operationalised an “interesting query” by quantifying when trending outcomes are more likely to occur (by month and day of the week).

A single Data Flow Task was implemented in the Control Flow panel. The workflow commenced with an OLE DB Source (get trend-flop) from trend-flop table (result from Assignment 11). A Lookup transformation against the Date dimension (date) was then applied, including month, month name and the day. Subsequently, a Conditional Split was used to produce a cleaned stream by excluding rows with null values of month and date_id = -1. A Multicast transformation then duplicated the cleaned stream into two parallel branches. In the first branch, an Aggregate (summary) transformation computed the metrics by grouping records by Month, month name, producing *TrendingCount* and *TotalReleased*. A Derived Column transformation was then applied to compute monthly summary for each month. For the second branch, aggregation (summary) was performed to compute a daily summary. The two streams (detailed daily results and monthly summary rows) were subsequently consolidated through a Union All. A further Derived Column was applied to format the percentage for reporting consistency. A final Conditional Split was used to remove any remaining invalid rows (null values for the day). Finally, the output was sorted by month and the day and written to the target destination via an OLE DB Destination.

This query is strategically important because it transforms a model-derived signal (trending versus flop) into an operational calendar that can inform release and promotion decisions. In a music business context, release timing is not merely administrative; it is typically interdependent with playlist refresh cycles, social-media consumption patterns, weekend listening behaviour, and competitive release congestion. Consequently, estimating the probability of trending outcomes conditional on month and weekday provides actionable guidance for planning and resource allocation.

Assignment 13

Query: Does “explicit” content affect a song’s chance of becoming Trending or Flopping, and does this effect change across lyric categories?

We started from the `Published_song_fact` source selected `song_id`, `category_id` and `explicit`. We then performed a Lookup to join with the `Trend_Flop` table, which is based on the results of **Assignment 11**, matching on `song_id` to retrieve `IsTrending` and `IsFlopping` indicators to each song. Next, we used Lookup on the category table, matching on `category_id`, to retrieve `lyrics_cat`.

After that we created an explicit tag using a Derived Column, assigning value 1 to explicit songs and 0 to non-explicit songs. We then applied an Aggregate transformation, grouping by `lyrics_cat` and `explicit tag`, and computed the total number of songs, the total number of trending songs, and the total number of flopping songs. Finally, we calculated the trending and flopping percentages and sorted the results.

We proposed this query because swear words and explicit language are extremely common in rap and hip-hop music as part of the genre identity and emotional intensity. Streaming platforms flag songs as *explicit* and apply restrictions such as parental filters, age limits, exclusion from some editorial playlists, reduced visibility in family accounts, etc. For a record label, this means that lyrical content directly affects potential reach and revenue. By analyzing whether explicit songs are more likely to become Trending or Flopping, and how this relationship changes across lyric categories, the record label can make informed strategic decisions. These insights help the record label guide artists toward content that performs better within their genre, reduce the risk of investing in songs with limited commercial potential, decide when to release non explicit versions to expand audience reach, and adapt release and marketing strategies to different markets and platforms.

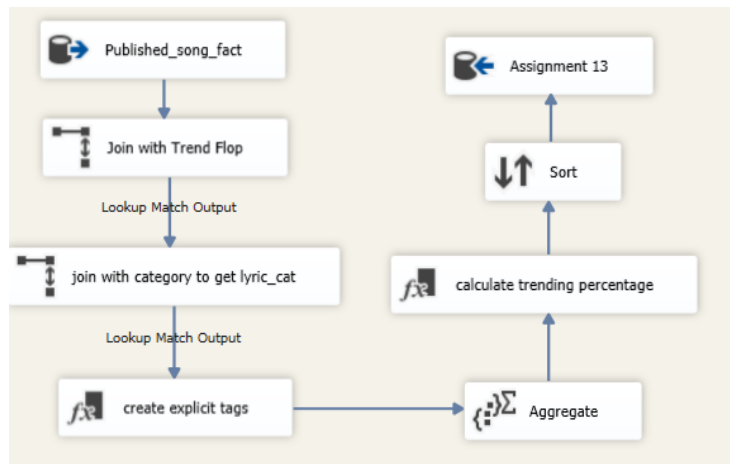


Figure 10: Assignment 13 data flow

Chapter 3: Using MDX, SQL Management Studio, and PowerBI

Assignment 14

To build the cube required for executing MDX queries, we used **SQL Server Analysis Services (SSAS)** and created a *Data Source View* from our data source, including the `Published_song_fact` table and all related dimension tables. Within the Data Source View, we introduced NAMED CALCULATIONS such as *Age* and *Age Range* in the *Artist* table, as well as additional temporal attributes in the *Date* table (*Seasons*, *SeasonYear*, *MonthYear*, and *DayofWeekSort*) to support demographic and time-based analyses.

The cube was designed with two measure groups. *The Published Song Fact* measure group contains the main analytical measures, namely *Stream 1month*, *Duration*, and *Published Song Fact Count*. The *Song Artist* measure group includes the measure *NArtist per song*, which counts the number of artists associated with each song and enables the distinction between solo and collaborative tracks in MDX queries.

Several dimensions were defined, including Album, Category, Artist, Date, and Artist Role, as well as two geographic dimensions Artist Geography B and Artist Geography C, representing the artist's birthplace and current location. The Published Song Fact dimension was created to correctly handle the many-to-many relationship between songs and artists. To structure the data effectively, a temporal hierarchy was defined in the Date dimension (*Year* → *SeasonYear* → *MonthYear* → *Date Id*), while spatial hierarchies (*Country* → *Region* → *Province* → *City* → *Artist Geo Id*) were defined in the Artist Geography dimensions.

Assignment 15

We queried the OLAP cube to analyze the monthly distribution of music streams at both regional and national levels. Using the *Stream 1month* measure, we aggregated the total number of streams for each month according to the artist's current geographic hierarchy, including both region-level values and country-level totals, while excluding unknown geographic members and records with null or empty month names. The analysis was restricted to songs in which the artist plays a primary role, ensuring that streams are not double counted due to featured collaborations. The resulting output reports, for each month, the total streams for every region together with the corresponding overall country total.

		Stream 1month
January		576141
January	Calabria	97561
January	Campania	1990125
January	Emilia-Romagna	148247
January	Lazio	2405969
January	Liguria	1265011
January	Lombardia	8307382
January	Marche	83790
January	Piemonte	281948
January	Puglia	351294
January	Sardegna	566708
January	Sicilia	1033962
January	Toscana	72604
January	Veneto	1679765
January	Italia	18860507

Figure 11: Assignment 15 result

Assignment 16

To compute the *average yearly artist streams*, we implemented a sequence of MDX SETS and CALCULATED MEMBERS. First, we defined the set CurrentSongs using EXISTING to consider only the songs present in the current evaluation context avoiding incorrect stream attribution when allocating streams by role. We then split CurrentSongs into SoloSet and CollabSet using the measure *NArtist per song*: songs with exactly one associated artist were classified as solo, while those with more than one artist were classified as collaborations. Next, we created calculated members to compute stream contributions: Solo Streams sums up *Stream 1month* over solo songs, while Primary Collab Allocated and Featured Collab Allocated allocate collaboration streams by role, weighting primary artists by 0.8 and featured artists by 0.2. These components were summed into *Total Allocated Streams*. We then defined *Grand Total Year Stream* as the total *Stream 1month* in the same year aggregated across all artists and finally computed Average yearly artist streams as the ratio between *Total Allocated Streams* and *Grand Total Year Stream* with the format string property set as "Percent". The final reports these measures on columns and lists year and artist identifiers on rows.

			Solo Streams	Primary Collab Allocated	Featured Collab Allocated	Total Allocated Streams	Grand Total Year Stream...	Average Yearly Artist Stream
1955	54	lazza	4756	(null)	(null)	4756	4756	100.00%
1970	30	entics	4242	(null)	(null)	4242	4242	100.00%
1972	46	il tre	5628	(null)	(null)	5628	5628	100.00%
1973	88	samuel heron	4911	(null)	(null)	4911	4911	100.00%
1976	29	ensi	5549	(null)	(null)	5549	5549	100.00%
1979	34	fedez	(null)	3304	(null)	3304	9224	35.82%
1979	52	johnny marsiglia	5094	(null)	(null)	5094	9224	55.23%
1979	1361	Cili Man	(null)	(null)	826	826	9224	8.95%
1979	1362	Sbend	(null)	(null)	826	826	9224	8.95%
1979	1363	Dani Mc	(null)	(null)	826	826	9224	8.95%
1983	80	rancore	5314	(null)	(null)	5314	5314	100.00%
1990	41	ghemon	5100	(null)	(null)	5100	5100	100.00%
1992	1	99 posse	8892	(null)	(null)	8892	57285	15.52%
1992	5	articolo 31	5349	(null)	(null)	5349	57285	9.34%
1992	8	bassi maestro	33037	4076	(null)	37113	57285	64.79%
1992	36	frankie hi-nrg mc	4912	(null)	(null)	4912	57285	8.57%
1992	763	Papa Ciccio	(null)	(null)	1019	1019	57285	1.78%
1993	1	99 posse	64285	10333.6	(null)	74618.6	268643	27.78%
1993	5	articolo 31	67108	8884	(null)	75992	268643	28.29%

Figure 12: Assignment 16 result

Assignment 17

The solution was implemented using MDX through a WITH clause that defined three calculated members, followed by a SELECT statement that returned the required measures across a two-dimensional result set (*category* × *year*). First, we calculated *yearly_stream*, was defined to represent the stream volume for the current evaluation context (that is, the currently selected *category* and *year*). Then, we calculated *prev_year_stream*, was defined using the LAG function on the current member of the *Date hierarchy*. After that, we assign ratio as 0 as follow:

$$\text{ratio} = \begin{cases} 0, & \text{if } \text{yearly_stream} = 0 \text{ or } \text{prev_year_stream} = 0 \text{ or either value is empty} \\ \frac{\text{yearly_stream} - \text{prev_year_stream}}{\text{prev_year_stream}}, & \text{otherwise} \end{cases}$$

Figure 13: Assignment 17 metric

A percentage format string was applied (FORMAT = "Percent"), so that the output is displayed as a percentage rather than a raw decimal fraction. The result is presented below:

		yearly_stream	prev_year_stream	ratio
The Complex	1992	15699	(null)	0.00%
The Complex	1994	20249	(null)	0.00%
The Complex	1995	9452	20249	-53.32%
The Complex	1996	21314	9452	125.50%
The Complex	1997	5141	21314	-75.88%
The Complex	1998	34005	5141	561.45%
The Complex	1999	24683	34005	-27.41%
The Complex	2000	31854	24683	29.05%

Figure 14: Assignment 17 result

Assignment 18

Current_Cat_Stream was defined as the base measure *Stream 1month*, evaluated at the current tuple (*Year* × *Season* × *Category*). Then, *Avg_All_Cats_Current* was defined as the mean of *Stream 1month* across all melody categories, while preserving the current date context (*Year* and *Season*). Next, *Prev_Year_Benchmark* was designed to represent the *previous-year value* of the seasonal *category-average benchmark*. This was achieved by shifting the time context back by one year using PARALLELPERIOD, while retaining the current season member. Finally, *Growth_Vs_Avg_Pct* was defined to express the relative increase of the current category stream above the previous-year seasonal benchmark:

$$\text{Growth_Vs_Avg_Pct} = \frac{\text{Current_Cat_Stream} - \text{Prev_Year_Benchmark}}{\text{Prev_Year_Benchmark}}$$

Figure 15: Assignment 18 metric

An IIF condition was then introduced to avoid division by zero and to handle empty benchmarks. Finally, the output is formatted as a percentage (FORMAT_STRING = "Percent"). The result is presented below:

			Current_Cat_Stream	Avg_All_Cats_Current	Prev_Year_Benchmark	Growth_Vs_Avg_Pct
1996	Spring	The Complex	5788	53327.75	4970.5	16.45%
1996	Spring	The High Energy	10145	53327.75	4970.5	104.10%
1996	Spring	The Mellow	26394	53327.75	4970.5	431.01%
1996	Spring	The Pure Tonal	170984	53327.75	4970.5	3339.98%
1996	Summer	The Complex	15526	32191.75	9788	58.62%
1996	Summer	The Mellow	29441	32191.75	9788	200.79%
1996	Summer	The Pure Tonal	74340	32191.75	9788	659.50%
1996	Winter	The Pure Tonal	123408	44282.3333333333	5620	2095.87%

Figure 16: Assignment 18 result

Query: Identify Top 5 Greatest of All Time Artists by Age Range and Melodic Category

To answer this query, we used the *Total Streams* measure, which represents the artist's overall contribution, as the ranking criterion. This measure is computed using the same SETS and CALCULATED MEMBERS defined in Assignment 16. The sets of melodic categories and age ranges were first generated, excluding empty members. For each category–age combination, the TOPCOUNT function was applied to rank artists by *Total Streams*, and the operation was repeated for all segments using the GENERATE function. Finally, NONEMPTY was used to remove empty results. The output reports *Total Streams* in the columns and lists melodic categories, age range, and artist name in the rows, showing the top performers.

			Total Streams
The High Energy	20-29	geolier	1693271.6
The High Energy	20-29	capo plaza	1150551.2
The High Energy	20-29	rosa chemical	602269.8
The High Energy	20-29	il tre	492595.4
The High Energy	30-39	gemitaiz	3208967.4
The High Energy	30-39	tedua	3193341
The High Energy	30-39	lazza	2430999.8
The High Energy	30-39	emis killa	1988885.2
The High Energy	30-39	sfera ebbasta	1916138.8
The High Energy	40-49	marracash	3332328.4
The High Energy	40-49	salmo	2434816
The High Energy	40-49	guè pequeno	2079431.6
The High Energy	40-49	fabri fibra	2022778
The High Energy	40-49	luchè	1896302.2
The High Energy	50-59	caparezza	1839178.8
The High Energy	50-59	j-ax	783836.4
The High Energy	50-59	bassi maestro	368660.6
The High Energy	50-59	don joe	301254.8
The High Energy	50-59	neffa	248289.4

Assignment 20



15

First, a column chart showing the total streams by city, broken down by four melody categories. This visualization allows a direct comparison of streaming volumes across artist birthplaces while highlighting the relative popularity of each category per city. Notably, Milano stands out as the city with the highest total stream count, with significant contributions from *The Mellow* and *The High Energy* categories (the graph can also visualize data with region, province or country by changing hierarchy). Second, a ring chart summarizing the total stream count by melody category across all cities. According to the data, *The Mellow* is the most streamed category overall, accounting for approximately 43.25% of the total. Next, a map visualization displays the total stream count by region, highlighting spatial patterns in artist origin and listener interest. The map indicates that northern and central Italy, particularly *Lombardy (Milano)* and surrounding areas, are the primary hubs for high-stream artists (the graph can also visualize data with region, province or country by changing hierarchy).

Assignment 21

First, a clustered column chart that shows total streams by region and lyrics category. This visualization highlights that *Lombardia and Campania* are the leading regions in terms of total streams. Almost all regions are dominated by *Italian Rap Life* (the graph can also visualize data with region, province or country by changing hierarchy). Second, a line chart illustrating total streams by day of the week reveals a striking peak on *Friday and Tuesday*, with a dramatic surge in the *Italian Rap Life* category. For other genres, high stream volumes are only recorded when debuting on *Friday*. Next, a seasonal chart further reinforces the observation about online music listening behavior, especially for *Italian Rap Life*, showing that *autumn* and *spring* are the most active seasons, while *summer* sees a significant drop in activity, possibly due to changes in habits or holidays. For *Neapolitan Rap*, *summer* debuts are more popular. Then, a pie chart offers a high-level overview of total streams by lyrics category. The dominance of *Italian Rap Life* is evident, accounting for over 92% of all streams, while other categories represent only minor fractions. A bar chart presents the published song count by *age range and lyrics category*. The highest number of published tracks belongs to the *30–39 and 40–49 age groups*. Finally, a weekly line graph tracks streaming trends across the year, identifying several peaks in weekly activity for each kind of lyrics category.



Figure 19: Assignment 21 result

Assignment 22

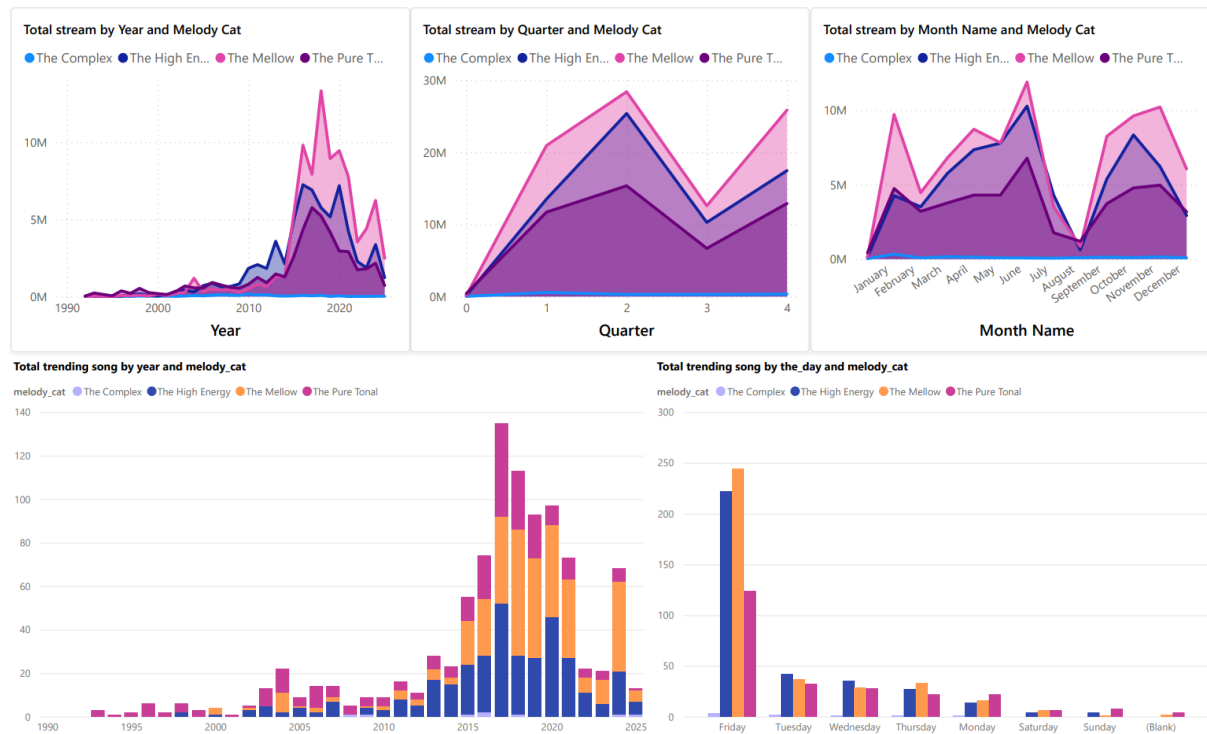


Figure 20: Assignment 22 result

An area chart of total streams by year, quarter, month and melody category illustrates the evolution of listening preferences. Notably, all categories show a sharp rise beginning around 2010, with *The Pure Tonal* and *The High Energy* experiencing significant peaks between 2015 and 2020. This suggests a cultural shift towards more melodic and emotionally resonant music styles in recent years. The second and four quarters consistently show higher stream volumes across others. The monthly breakdown further refines this observation, with *January*, *June*, *October* and *November* standing out as peak months. Next, a bar chart displaying the number of trending songs by year and melody category provides a view of content production or popularity trends. From around 2014 to 2022, there is a surge in the number of trending songs, with *The High Energy* consistently contributing the highest count. However, there has also been a shift in trend with a sharp increase in the number of songs by *The Mellow* and *The Pure Tonal* since 2014. The final bar chart segments trending songs by day of the week, revealing that *Friday* is by far the most active day for trending songs across all melody categories.